

# **“Product Search and Recommendation System”**

*Project Report submitted for the partial fulfilment of grade for the course*

**Data Structures and Algorithms -3 (25CS2103E)**

**in**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**by**

**T. HIMABINDU SREE**

**:2520030484**

**Team Number: 4**

**Section: 2**

Under the Guidance of

**Dr. S. Vinay Kumar**

**Associate Professor**

**Department of Computer Science and Engineering**



**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**2026-27**

# Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

## DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

### Declaration

The Project Report entitled “**Product Search and Recommendation System**” is a record of Bonafide work of **T. Himabindu Sree – 2520030484** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

**T. Himabindu Sree – 2520030484**

# Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

## DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

### CERTIFICATE

This is certify that the mini project based report entitled **“Product Search and Recommendation System”** is a bonafide work done and submitted by **T. HIMABINDU SREE – 2520030484** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27**.

**Signature of the Guide**  
(Dr. S. Vinay Kumar)

**Signature of the HOD**

**Signature of the External Examiner**

## **ACKNOWLEDGEMENT**

I express my sincere gratitude to my guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance, constant encouragement and constructive suggestions throughout this project. His clear explanations of algorithms and his insistence on testing every claim helped me to complete this work in a systematic manner.

I am also thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and the academic environment needed to carry out this project work.

Finally, I thank my faculty members, friends and family for their support and motivation during the course Data Structures and Algorithms-3 (25CS2103E).

**T. Himabindu Sree – 2520030484**



## **Abstract**

Online shoppers search large product catalogues with short, often misspelled queries and expect the most relevant items first, together with a short list of sensible alternatives. This project, titled “Product Search and Recommendation System”, is a console-based Java application that solves this problem using classical data structures and algorithms taught in the course Data Structures and Algorithms-3. The catalogue is a corpus of 300 product records stored as plain text files across eleven categories.

The system combines five algorithms. Knuth–Morris–Pratt (KMP) and Rabin–Karp perform exact phrase matching, and both are also used in a /time benchmark that compares them on the same corpus. Levenshtein edit distance recovers from spelling mistakes before searching, for example “samsng phone” is corrected to “samsung phone”. A Trie provides prefix suggestions for the /trie command. A transparent scoring model ranks the results. Finally, the Edmonds–Karp maximum-flow algorithm selects up to six recommended products so that the list respects limits on the total number of products, products per category and products per brand.

The implementation was verified by compiling it without warnings and by running an automated suite of 184 checks, all of which pass. The algorithms were additionally compared with independent reference implementations on tens of thousands of random cases. Results show that KMP and Rabin–Karp always agree on match counts, that the recommendation lists never violate their limits, and that on the 300-file corpus KMP scans took under 6 ms against about 10–12 ms for the Rabin–Karp implementation used. The report documents the requirements, design, pseudocode, results, the defects found and fixed during review, and the limitations of the system.

# Table of Contents

|   |                                                           | Page No. |
|---|-----------------------------------------------------------|----------|
|   | Acknowledgement                                           |          |
|   | Abstract                                                  |          |
|   | Table of Contents                                         |          |
|   | List of Figures                                           |          |
| 1 | Introduction                                              | 1–3      |
|   | 1.1 Overview                                              |          |
|   | 1.2 Problem Statement                                     |          |
|   | 1.3 Scope of the Project                                  |          |
|   | 1.4 Objectives                                            |          |
| 2 | System Requirements                                       | 4–5      |
|   | 2.1 Functional Requirements                               |          |
|   | 2.2 Non-Functional Requirements                           |          |
|   | 2.3 Related Work and Existing Approaches                  |          |
|   | 2.4 Hardware and Software Specifications                  |          |
| 3 | System Design                                             | 6–9      |
|   | 3.1 High-level Architecture Diagram                       |          |
|   | 3.2 Description of Each Layer                             |          |
|   | 3.3 Dataset (Corpus) Design                               |          |
|   | 3.4 Algorithm Selection and Rationale                     |          |
|   | 3.5 Search and Ranking Design                             |          |
|   | 3.6 Recommendation Network Design                         |          |
| 4 | Implementation                                            | 10–15    |
|   | 4.1 Module-wise Implementation                            |          |
|   | 4.2 Console Interface Logic and State Handling            |          |
|   | 4.3 Integration of the Modules                            |          |
|   | 4.4 Testing Implementation                                |          |
| 5 | Results                                                   | 16–21    |
|   | 5.1 Test Environment and Method                           |          |
|   | 5.2 Search and Spelling Recovery Results                  |          |
|   | 5.3 Recommendation Results                                |          |
|   | 5.4 Trie Prefix Search Results                            |          |
|   | 5.5 KMP versus Rabin–Karp Benchmark                       |          |
|   | 5.6 Correctness Verification                              |          |
| 6 | Challenges & Limitations                                  | 22–23    |
|   | 6.1 Issues Faced During Development                       |          |
|   | 6.2 Solutions Applied                                     |          |
|   | 6.3 Limitations of the System                             |          |
| 7 | Conclusion & Future Work                                  | 24–24    |
|   | 7.1 Conclusion                                            |          |
|   | 7.2 Future Work                                           |          |
|   | 7.3 Skills Learned During Development                     |          |
|   | References                                                | 25       |
|   | Books, journal papers and documentation cited in the text |          |

## List of Figures

|     |                                                                                                          |    |
|-----|----------------------------------------------------------------------------------------------------------|----|
| 1.1 | Block diagram of the Product Search and Recommendation System .....                                      | 2  |
| 3.1 | Layered architecture of the system .....                                                                 | 6  |
| 3.2 | Flow chart of one search with spelling recovery and fallback .....                                       | 8  |
| 3.3 | Flow network used for recommendations (default limits shown) .....                                       | 9  |
| 5.1 | Console output: start-up (300 files loaded) and the search “samsng phone” with spelling correction ..... | 17 |
| 5.2 | Recommendation engine output (Edmonds–Karp) for “samsng phone” .....                                     | 18 |
| 5.3 | Trie prefix search for the prefix “app” .....                                                            | 19 |
| 5.4 | Console output of the /time benchmark for the pattern “5g” .....                                         | 20 |
| 5.5 | Average scan time of KMP and Rabin–Karp over the corpus (n = number of occurrences) .....                | 20 |
| 5.6 | Automated test suite result (TestRunner) .....                                                           | 21 |

# 1. INTRODUCTION

## 1.1 Overview

Searching is the most common way in which people find products online. A shopper types a few words such as “samsung phone” and expects a ranked list of relevant products, tolerance to typing mistakes and a few additional suggestions. Such systems belong to the field of information retrieval [1], and their quality depends on the data structures and algorithms behind them: how text is matched, how errors are corrected, how prefixes are completed and how a short list is chosen from many candidates.

This project builds a console-based Product Search and Recommendation System in Java over a corpus of 300 product records stored as text files. Five classical algorithms are integrated: Knuth–Morris–Pratt (KMP) [2] and Rabin–Karp [3] string matching, Levenshtein edit distance [4], the Trie (prefix tree) [5] and the Edmonds–Karp maximum-flow algorithm [6], a refinement of the Ford–Fulkerson method [7]. A general treatment of all five is given in [8].

Fig. 1.1 shows the overall structure. The query is spell-corrected with Levenshtein distance, matched and scored using whole-word matching and KMP or Rabin–Karp phrase matching, and the best ten results are printed. On request, the top 30 results go to a flow network solved by Edmonds–Karp, which returns up to six products that respect the diversity limits. Two more commands expose the Trie (/trie) and the KMP versus Rabin–Karp benchmark (/time).

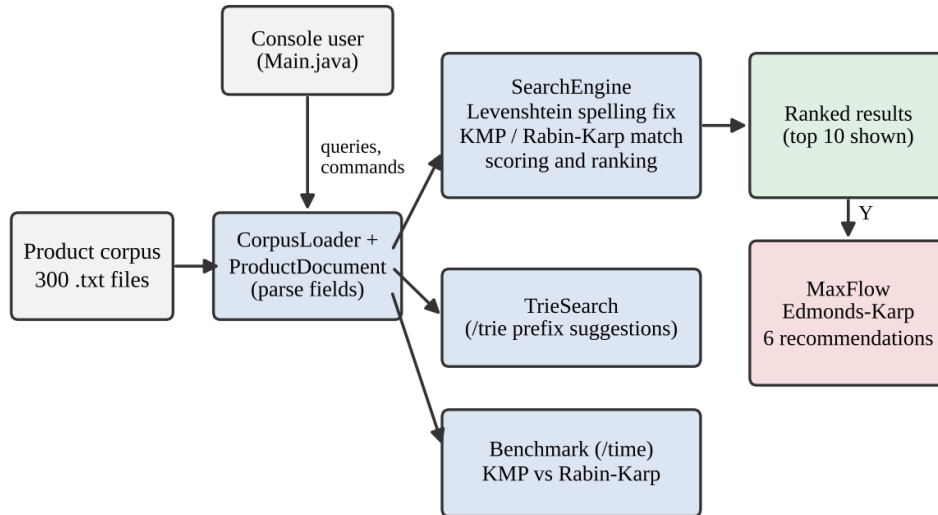


Fig. 1.1 Block diagram of the Product Search and Recommendation System

## 1.2 Problem Statement

A catalogue stored as many separate text files cannot be searched sensibly with a simple substring test: short queries such as “n” match unrelated text, misspelled words (“samsng”) return nothing, prefixes (“app”) need fast completion, and a ranked list full of one brand gives the user little choice. The problem addressed here is to *design and implement a console application that, for 300 product records, (a) finds and ranks products for a free-text query using exact pattern matching, (b) recovers from spelling mistakes, (c) suggests products from a typed prefix, and (d) recommends at most six products subject to total, per-category and per-brand limits, while verifying the correctness and performance of the algorithms.*

## 1.3 Scope of the Project

- A corpus of 300 unique product records in eleven categories (Smartphone, Laptop, Audio, Monitor, Wearable, Accessories, Storage, Camera, Networking, Tablet, Printer).
- A console interface only; no graphical interface, database or network access.
- English queries of one or more words, normalised to lower case with punctuation treated as a space.

- Ranked search, prefix search, pattern-matching benchmark and a six-product recommendation list, with automated verification of every algorithm.

Personalised recommendation (user profiles, purchase history), semantic search and verification of specifications against manufacturers are outside the scope. The product data is illustrative sample data.

## 1.4 Objectives

1. Implement KMP and Rabin–Karp once, in a shared class used by both search and benchmark.
2. Implement Levenshtein distance with linear extra space for spelling recovery and approximate search.
3. Implement a Trie that returns all products having a word that starts with a typed prefix.
4. Model the recommendation limits as a flow network and solve it with Edmonds–Karp.
5. Provide an explainable ranking model and keep the corpus consistent (exactly 300 files).
6. Test every algorithm against an independent reference and report only executed results.

The remaining chapters cover requirements (Chapter 2), design (3), implementation in pseudocode (4), results (5), challenges and limitations (6), and conclusion with future work (7).

## 2. SYSTEM REQUIREMENTS

### 2.1 Functional Requirements

**Table 2.1 Functional requirements**

| ID   | Requirement                                                                                                                          | Module                        |
|------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| FR1  | Load all .txt files of the corpus folder and show the number actually loaded.                                                        | CorpusLoader                  |
| FR2  | Parse name, brand, category, price, rating and search text of each record.                                                           | ProductDocument               |
| FR3  | Accept a free-text query; handle empty or punctuation-only input without crashing.                                                   | Main, SearchEngine            |
| FR4  | Correct misspelled query words with Levenshtein distance and show the change.                                                        | SearchEngine                  |
| FR5  | Match by whole word and by exact phrase (KMP or Rabin–Karp); rank and show the top 10 and the total.                                 | SearchEngine, PatternMatching |
| FR6  | If nothing matches, try an approximate product-level search, else report no match.                                                   | LevenshteinSearch             |
| FR7  | On request, recommend up to six products (at most 6 per category, 2 per brand) and show the augmenting paths and a constraint check. | MaxFlow                       |
| FR8  | /trie: list products whose name or search index has a word starting with the prefix.                                                 | Trie, TrieSearch              |
| FR9  | /time: compare KMP and Rabin–Karp on the corpus and check that their counts agree.                                                   | Main, PatternMatching         |
| FR10 | exit (or end of input) closes the program quietly.                                                                                   | Main                          |

### 2.2 Non-Functional Requirements

**Table 2.2 Non-functional requirements**

| Attribute        | Requirement and how it is met                                                                                                                    |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Response time    | A query over 300 records responds without visible delay: fields and search text are parsed once and cached; a KMP scan takes a few milliseconds. |
| Usability        | Plain console menu, readable tables, a “Did you mean” line and helpful error messages.                                                           |
| Reliability      | No crash on empty input, a missing corpus folder or end of input; covered by tests.                                                              |
| Correctness      | Algorithms agree with independent references (184 automated checks).                                                                             |
| Scalability      | Search cost is linear in the corpus size; the Trie and the flow network are small.                                                               |
| Portability      | Pure Java with ASCII-only sources; compiles with --release 8 to 21.                                                                              |
| Data consistency | One spelling per brand, a valid category, no duplicates; enforced on every test run.                                                             |

## 2.3 Related Work and Existing Approaches

Exact string matching has a long history. The naive method can take  $O(nm)$  time for a text of length  $n$  and a pattern of length  $m$ . Knuth, Morris and Pratt avoid re-examining characters with a precomputed failure table, giving  $O(n + m)$  in the worst case [2]. Karp and Rabin compare rolling hash values of the pattern and of each window, which is  $O(n + m)$  in expectation,  $O(nm)$  in the worst case, and extends to many patterns [3]; textbook treatments are in [8] and [9]. Approximate matching uses the edit distance of Levenshtein [4], computed by dynamic programming [10] and surveyed for spelling correction in [11]. The Trie of Fredkin [5] stores words along paths so that all words of a prefix lie in one subtree. Recommender systems are dominated by collaborative-filtering methods that need user histories [12]; viewing a recommendation list as a selection problem under capacity limits leads directly to maximum flow [6], [7].

The gaps addressed here are: algorithms are often shown in isolation, whereas here all five cooperate in one pipeline; benchmark programs often print an “expected” count instead of each algorithm’s own count; recommendations are often just the top- $k$  results, whereas a flow network guarantees diversity limits; and correctness claims are backed by reproducible automated tests.

## 2.4 Hardware and Software Specifications

**Table 2.3 Hardware and software used**

| Item                    | Specification                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------|
| Processor, memory, disk | Any 64-bit CPU; 512 MB free RAM; about 10 MB disk (the corpus is under 5 MB)                        |
| Operating system        | Windows, Linux or macOS (measurements in this report: Linux container)                              |
| Language / JDK          | Java; compiles with --release 8, 11, 17, 21; run-tested on JDK 21 [13]                              |
| Compiler flags          | javac -encoding UTF-8 -Xlint:all (no warnings)                                                      |
| Verification tools      | Python 3 (regular expressions, recursion), networkx (reference max flow), a brute-force recommender |
| Interface               | Terminal / command prompt (no GUI)                                                                  |

## 3. SYSTEM DESIGN

### 3.1 High-level Architecture Diagram

The system has four layers (Fig. 3.1). Upper layers call lower layers only, so each algorithm is independent of the console and the same class serves the application, the benchmark and the tests.

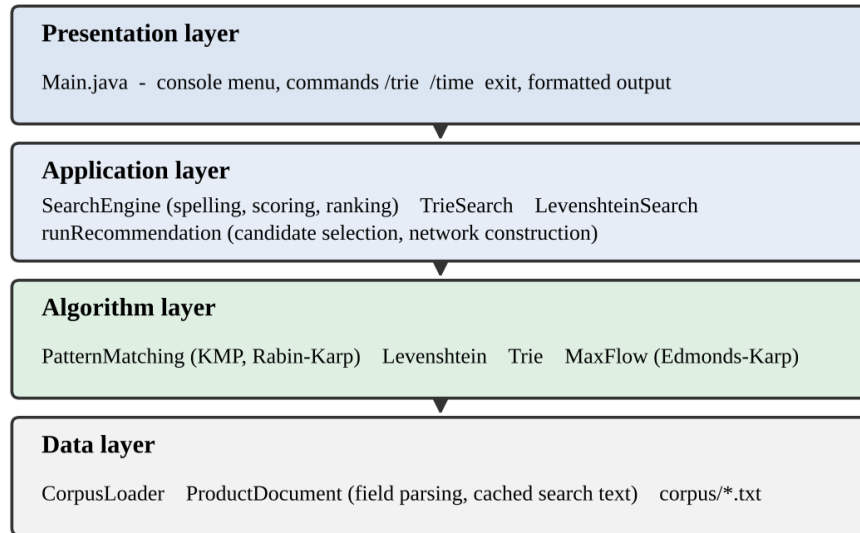


Fig. 3.1 Layered architecture of the system

### 3.2 Description of Each Layer

- **Presentation:** Main.java owns the console, prints the number of corpus files loaded (the length of the loaded array, never a constant) and dispatches the commands /trie, /time and exit.
- **Application:** SearchEngine builds an index of every product once and a word-frequency table for spelling recovery, and exposes rank(query) and search(query). TrieSearch, LevenshteinSearch and runRecommendation (in MaxFlow) complete this layer.
- **Algorithm:** PatternMatching (KMP, Rabin-Karp), Levenshtein, Trie and MaxFlow (Edmonds-Karp). None depends on the console.
- **Data:** CorpusLoader reads the .txt files in a deterministic (sorted) order as UTF-8; ProductDocument lazily parses and caches each record's fields.

### 3.3 Dataset (Corpus) Design

The corpus has exactly 300 files, one product per file. A record has product data (ID, name, brand, category, subcategory, price, rating, specifications), key features with search terms, and a search-index section with synonyms and customer queries. Table 3.1 gives the products per category.

**Table 3.1 Products per category in the corpus**

| Category   | Products | Category     | Products   |
|------------|----------|--------------|------------|
| Smartphone | 59       | Accessories  | 24         |
| Laptop     | 48       | Storage      | 23         |
| Audio      | 44       | Camera       | 22         |
| Monitor    | 28       | Networking   | 12         |
| Wearable   | 26       | Tablet       | 8          |
| Printer    | 6        | <b>Total</b> | <b>300</b> |

The data is sample data: 43 records (product\_301 to product\_343) were added to reach 300 files and use generic specification text with illustrative prices and ratings. The report therefore never presents the specifications as verified manufacturer data.

### 3.4 Algorithm Selection and Rationale

**Table 3.2 Algorithms, their use and their complexity**

| Algorithm             | Used for                    | Time                                            | Extra space             | Why suitable                   |
|-----------------------|-----------------------------|-------------------------------------------------|-------------------------|--------------------------------|
| KMP [2]               | Phrase matching, benchmark  | $O(n+m)$ worst case                             | $O(m)$                  | Guaranteed linear time         |
| Rabin–Karp [3]        | Phrase matching, benchmark  | $O(n+m)$ expected; $O(nm)$ worst                | $O(1)$                  | Rolling hash; many patterns    |
| Levenshtein [4], [10] | Spelling recovery, fallback | $O(a \cdot b)$                                  | $O(\min(a,b))$          | Counts insert, delete, replace |
| Trie [5]              | Prefix suggestions          | $O(L)$ insert/find; $O(L + \text{output})$ list | $O(\text{total chars})$ | A prefix is one subtree        |
| Edmonds–Karp [6]      | Recommendations             | $O(V \cdot E^2)$ ; here at most 7 BFS passes    | $O(V+E)$                | Limits are capacities          |

Here  $n$  is the text length,  $m$  the pattern length,  $a$  and  $b$  word lengths,  $L$  a word length, and  $V$  and  $E$  the nodes and edges of the flow network. The maximum flow is at most 6, so each run needs at most six augmentations and one failed search, i.e. at most 7 breadth-first searches of  $O(V+E)$ .

### 3.5 Search and Ranking Design

Fig. 3.2 shows one search. The query is normalised; each word of three or more letters that is not in the corpus vocabulary is replaced by the closest corpus word; then every product is scored with the weights of Table 3.3. Matching is by whole word, tolerating a trailing model digit (“buds” matches “Buds3”), plurals and compound words (“phone” matches “iPhone”). Rating only breaks ties, so every score can be explained.

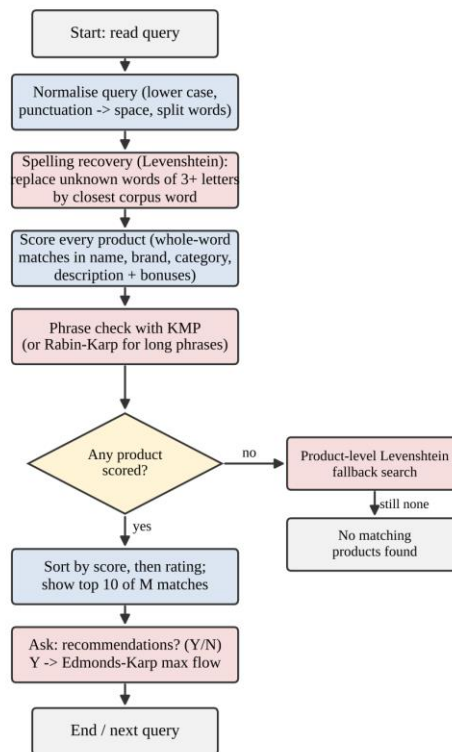


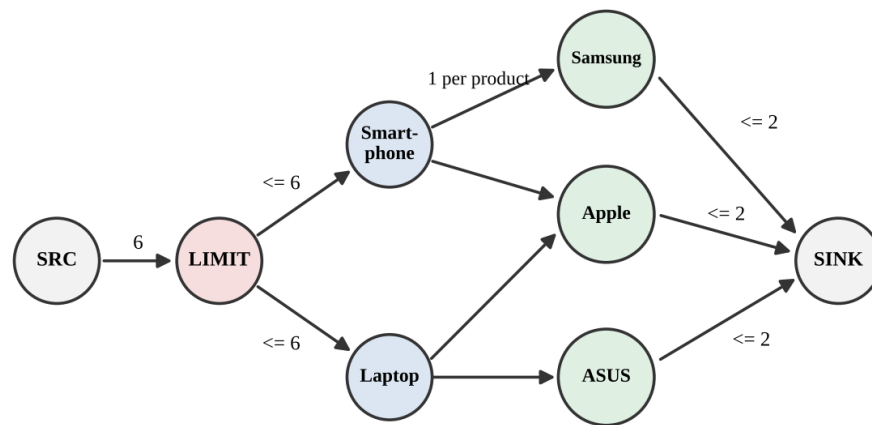
Fig. 3.2 Flow chart of one search with spelling recovery and fallback

**Table 3.3 Scoring weights used by the ranking model**

| Condition                                                      | Points        |
|----------------------------------------------------------------|---------------|
| Query word equals a word of the product name                   | +100 per word |
| Query word equals the brand                                    | +80 per word  |
| Query word equals the category (or a synonym)                  | +180 per word |
| Query word occurs in the description text                      | +10 per word  |
| Whole query equals the name / occurs as a phrase in the name   | +300 / +220   |
| All query words matched (some only in the description: +150)   | +200          |
| Query names a category: product is in it / is not in it        | +250 / -180   |
| Whole phrase confirmed by KMP or Rabin–Karp in the search text | +50           |

### 3.6 Recommendation Network Design

At most six products may be recommended, at most six per category and two per brand. These limits are edge capacities (Fig. 3.3). Flow leaves the SOURCE with capacity 6 into a LIMIT node; one edge per category with the per-category capacity follows; every candidate product is one edge of capacity 1 from its category node to its brand node; each brand node has an edge to the SINK with the per-brand capacity. One unit of flow is one recommended product, so the maximum flow is the largest list that breaks no limit.



Each product is one edge CATEGORY -> BRAND of capacity 1.  
Default limits: total 6, per category 6, per brand 2.

Fig. 3.3 Flow network used for recommendations (default limits shown)

Max flow gives the count, not the products. Products are therefore dropped from the lowest-ranked candidate upwards while the flow stays unchanged, which keeps the best-ranked products. If the default limits cannot fill six slots (for example the query “apple”, where all candidates share one brand), they are relaxed in two documented steps (2 per category and 6 per brand, then 6 and 6) and the console states which limits were used.

## 4. IMPLEMENTATION

This chapter explains each module with pseudocode that follows the Java classes closely; null checks and printing details are omitted. Strings are indexed from 0.

### 4.1 Module-wise Implementation

#### 4.1.1 Corpus loading and parsing (CorpusLoader, ProductDocument)

CorpusLoader lists the corpus folder, keeps the files ending in .txt, sorts them by name and reads each as UTF-8; a missing folder gives a hint instead of a stack trace. ProductDocument extracts each field only once and caches it, which cut a 400-query test sweep from 68 s to 4 s.

#### 4.1.2 Knuth–Morris–Pratt (PatternMatching.countKMP)

KMP builds the table  $lps$ , where  $lps[i]$  is the length of the longest proper prefix of  $P[0..i]$  that is also a suffix of it. After a mismatch following  $j$  matched characters the pattern moves to  $lps[j-1]$ , so the text pointer never moves backwards, and overlapping occurrences are counted [2].

##### Algorithm 4.1 KMP: count overlapping occurrences of P in T

```
function BUILD_LPS(P):  
   $lps[0..m-1] \leftarrow 0$ ;  $len \leftarrow 0$ ;  $i \leftarrow 1$   
  while  $i < m$  do  
    if  $P[i] = P[len]$  then  $len \leftarrow len + 1$ ;  $lps[i] \leftarrow len$ ;  $i \leftarrow i + 1$   
    else if  $len > 0$  then  $len \leftarrow lps[len - 1]$   
    else  $lps[i] \leftarrow 0$ ;  $i \leftarrow i + 1$   
  return  $lps$   
function COUNT_KMP(T, P):  
  if P is empty or  $length(T) < length(P)$  then return 0  
   $lps \leftarrow BUILD\_LPS(P)$ ;  $i \leftarrow 0$ ;  $j \leftarrow 0$ ;  $count \leftarrow 0$   
  while  $i < length(T)$  do  
    if  $T[i] = P[j]$  then  
       $i \leftarrow i + 1$ ;  $j \leftarrow j + 1$   
      if  $j = length(P)$  then  $count \leftarrow count + 1$ ;  $j \leftarrow lps[j - 1]$   
    else if  $j > 0$  then  $j \leftarrow lps[j - 1]$   
    else  $i \leftarrow i + 1$   
  return  $count$ 
```

#### 4.1.3 Rabin–Karp (PatternMatching.countRabinKarp)

Rabin–Karp compares a polynomial hash (base 256, modulus 1,000,000,007) of the pattern with that of the current window, updated in constant time. Equal

hashes are verified character by character, so a collision can never give a false match; this is why its count always equals the KMP count [3].

#### Algorithm 4.2 Rabin–Karp with verification

```

function COUNT RK(T, P):
  n ← length(T); m ← length(P); B ← 256; M ← 1000000007
  if m = 0 or n < m then return 0
  high ← B(m-1) mod M; hp ← 0; hw ← 0
  for i ← 0 to m - 1 do
    hp ← (hp * B + P[i]) mod M; hw ← (hw * B + T[i]) mod M
  count ← 0
  for i ← 0 to n - m do
    if hp = hw and T[i..i+m-1] = P then count ← count + 1
    if i < n - m then
      hw ← (hw - T[i] * high) mod M; if hw < 0 then hw ← hw + M
      hw ← (hw * B + T[i + m]) mod M
  return count

```

#### 4.1.4 Levenshtein distance and spelling recovery

The edit distance is computed row by row; row  $i$  depends only on row  $i-1$ , so two rows of length  $\min(a,b)+1$  suffice: time  $O(a \cdot b)$ , extra space  $O(\min(a,b))$  [4], [10]. Spelling recovery is deliberately conservative: words under three letters, words containing a digit (“s24”, “128gb”) and words already in the corpus vocabulary (“amoled”) are never rewritten. Otherwise the closest vocabulary word wins, ties go to the word found in more products and then to alphabetical order, and the suggestion is accepted only within distance 1 (words up to four letters) or 2 [11].

#### Algorithm 4.3 Levenshtein distance (two rows) and word correction

```

function DISTANCE(a, b):
  a ← lower(a); b ← lower(b); if length(b) > length(a) then swap(a, b)
  prev[j] ← j for j = 0..n // n = length(b), m = length(a)
  for i ← 1 to m do
    cur[0] ← i
    for j ← 1 to n do
      if a[i-1] = b[j-1] then cur[j] ← prev[j - 1]
      else cur[j] ← 1 + min(cur[j - 1], prev[j], prev[j - 1])
    swap(prev, cur)
  return prev[n]
function CORRECT(term):
  if length(term) < 3 or term has a digit or term in vocabulary then return term
  best ← none; bestD ← ∞
  for each (word, freq) in vocabulary with |length(word) - length(term)| ≤ 2 do
    d ← DISTANCE(term, word)
    if d < bestD or (d = bestD and better by freq, then alphabetically) then best ← word; bestD ← d
  limit ← 1 if length(term) ≤ 4 else 2
  if best ≠ none and bestD ≤ limit then return best else return term

```

#### 4.1.5 Scoring, ranking and fallback (SearchEngine.rank)

After spelling recovery every product is scored with the weights of Table 3.3. A de-duplicated copy of the query is used for the phrase check, so “phone phone” ranks like “phone”. Products with a positive score are sorted by score and then rating. If none scores, LevenshteinSearch tries an approximate product-level match whose threshold is zero for terms of one or two letters or with digits, so noise such as “n”, “tv” or “128” matches nothing.

#### Algorithm 4.4 Rank products for a query

```
function RANK(query):  
  terms ← [CORRECT(t) for each word t of normalise(query)]; if terms is empty  
  then return empty  
  intent ← category named by the terms, if any; results ← empty list  
  for each product p do  
    s ← 0  
    for each term t in terms do  
      if t is a whole-word match in name(p) then s ← s + 100  
      if t = brand(p) then s ← s + 80  
      if t matches category(p) then s ← s + 180  
      if t occurs in description(p) then s ← s + 10  
    s ← s + exact-name, phrase-in-name and all-terms bonuses  
    if intent ≠ none then s ← s + 250 if category(p) = intent else s - 180  
    if COUNT KMP(searchText(p), phrase) > 0 then s ← s + 50  
    if s > 0 then add (p, s) to results  
  sort results by score descending, then rating descending  
  if results is empty then results ← approximate Levenshtein search  
  return results
```

#### 4.1.6 Trie and prefix search (Trie, TrieSearch)

Trie nodes keep children in a sorted map, so completions come back alphabetically. TrieSearch inserts every word of the product name and of the Search Index section (at least two letters) and maps each word to its products with an O(1) duplicate check. The typed prefix is normalised like the index (“Galaxy-S2” works); for several words the product sets are intersected.

#### Algorithm 4.5 Trie insertion and prefix completion

```
function INSERT(word):
    node  $\leftarrow$  root
    for each character c of word do
        if node has no child c then create child c
        node  $\leftarrow$  child c
    mark node as end of word
function WORDS_WITH_PREFIX(prefix):
    node  $\leftarrow$  root
    for each character c of prefix do
        if node has no child c then return empty list
        node  $\leftarrow$  child c
    result  $\leftarrow$  empty list; COLLECT(node, prefix, result); return result
function COLLECT(node, text, result):
    if node is end of word then append text to result
    for each child (c, next) of node in alphabetical order do COLLECT(next, text + c, result)
```

#### 4.1.7 Edmonds–Karp maximum flow (MaxFlow.calculateMaxFlow)

The graph is stored as adjacency lists; every edge has a reverse edge of capacity 0 so that flow can be undone. Each iteration finds the shortest augmenting path from SOURCE to SINK by breadth-first search, pushes the bottleneck amount and updates the residual capacities. Choosing the shortest path bounds the number of augmentations by  $O(VE)$  and the total time by  $O(VE^2)$  [6], [8].

#### Algorithm 4.6 Edmonds–Karp

```
function MAX_FLOW(source, sink):
    flow  $\leftarrow$  0
    while true do
        parent[*]  $\leftarrow$  -1; parent[source]  $\leftarrow$  source; queue  $\leftarrow$  [source]
        while queue not empty and parent[sink] = -1 do
            u  $\leftarrow$  dequeue(queue)
            for each edge e = (u, v) with capacity(e) > 0 and parent[v] = -1 do
                parent[v]  $\leftarrow$  (u, e); enqueue(queue, v)
            if parent[sink] = -1 then break // no augmenting path left
            push  $\leftarrow$  minimum capacity along the path sink  $\rightarrow$  source
            for each edge e on the path do
                capacity(e)  $\leftarrow$  capacity(e) - push; capacity(reverse(e))  $\leftarrow$ 
                    capacity(reverse(e)) + push
            flow  $\leftarrow$  flow + push
    return flow
```

#### 4.1.8 Recommendation selection and benchmark

The search engine passes the 30 best results with category, brand and score. Duplicate names are removed, categories and brands are numbered case-insensitively, and a product without a brand gets a brand of its own so that a missing brand never limits anything. The network of Section 3.6 is solved for each limit tier

until one reaches six products, then candidates are dropped from the bottom of the ranking while the flow is unchanged (Algorithm 4.7). The /time benchmark normalises pattern and text in the same way (“USB-C” becomes “usb c”), runs three warm-up and ten measured rounds per algorithm in alternating order, requires equal counts in every round, prints each algorithm’s own count and declares no winner when the averages differ by less than 5%.

#### Algorithm 4.7 Choose the recommended products

```

function RECOMMEND(candidates, total = 6):
    tiers  $\leftarrow$  [(6, 2), (2, 6), (6, 6)] // (per category, per brand)
    target  $\leftarrow$  min(total, |candidates|); best  $\leftarrow$  -1
    for k  $\leftarrow$  0 to 2 do
        f  $\leftarrow$  BUILD_AND_RUN(candidates, none removed, total, tiers[k])
        if f > best then best  $\leftarrow$  f; used  $\leftarrow$  k
        if f  $\geq$  target then break
    removed[*]  $\leftarrow$  false
    for i  $\leftarrow$  |candidates| - 1 downto 0 do // lowest ranked first
        removed[i]  $\leftarrow$  true
        if BUILD_AND_RUN(candidates, removed, total, tiers[used]) < best then
            removed[i]  $\leftarrow$  false
    return all candidates with removed[i] = false
function BUILD_AND_RUN(cands, removed, total, (perCat, perBrand)):
    add SOURCE  $\rightarrow$  LIMIT (total); LIMIT  $\rightarrow$  each category (perCat)
    for each live candidate p: add category(p)  $\rightarrow$  brand(p) with capacity 1
    for each brand b: add b  $\rightarrow$  SINK (perBrand)
    return MAX_FLOW(SOURCE, SINK)

```

## 4.2 Console Interface Logic and State Handling

The system is a console application, so there is no front-end rendering or API; the interface is a loop with little state. It reads a line, trims it and dispatches: exit ends the loop, /trie and /time start their own prompts, an empty or punctuation-only line prints a message, and anything else is a search. After a search with results the program asks for recommendations and accepts Y or yes in any case. Every read is guarded by hasNextLine, so the end of input (for example a piped file) stops the program quietly.

## 4.3 Integration of the Modules

ProductDocument is the single data carrier used by the search engine, the Trie and the benchmark. PatternMatching is the single implementation of KMP and Rabin–Karp: SearchEngine uses it for the phrase bonus and Main uses it for the benchmark. Levenshtein serves both spelling recovery and the fallback search (an earlier private duplicate was removed). MaxFlow receives only plain arrays (names,

categories, brands, scores), which allowed it to be tested against a brute-force reference.

## **4.4 Testing Implementation**

TestRunner prints a header per group of checks, counts passes and failures and returns exit code 0 only when all checks pass. Its groups are KMP and Rabin–Karp (randomised against `String.indexOf`), Levenshtein (against a recursive definition), Trie, Edmonds–Karp (against brute-force minimum cut), recommendation limits and tiers, corpus consistency, parsing, search ranking and spelling, the console flow, Trie search, fallback search and benchmark counts. TrieTest is a small self-checking Trie demonstration.

## 5. RESULTS

### 5.1 Test Environment and Method

All results were produced by compiling with `javac -Xlint:all -encoding UTF-8` (no warnings) and running on JDK 21 on Linux. Search, recommendation and Trie outputs come from real console sessions. Timings depend on the machine and JVM warm-up; match counts are deterministic. The screenshots in this chapter show the same commands on the author’s machine.

### 5.2 Search and Spelling Recovery Results

**Table 5.1 Sample queries and observed behaviour**

| Query             | Corrected query               | Matches | Result 1                            |
|-------------------|-------------------------------|---------|-------------------------------------|
| samsng phone      | samsung phone (distance 1)    | 77      | Samsung Galaxy S24 Ultra (880)      |
| galaxy s24 ultr   | galaxy s24 ultra (distance 1) | 28      | Samsung Galaxy S24 Ultra (800)      |
| headphons         | headphones (distance 1)       | 44      | Sony WH-1000XM6 Wireless Headphones |
| laptop            | no change                     | 48      | Framework Laptop 16 (810)           |
| phone             | no change                     | 59      | Asus ROG Phone 9 Pro (810)          |
| apple             | no change                     | 21      | Apple iPhone 16 Pro                 |
| waterproof        | no change                     | 3       | Apple AirPods Pro (2nd Gen USB-C)   |
| xyznonexistent123 | no change                     | 0       | No matching products found          |

The first row shows the whole pipeline: Levenshtein corrects the query and the six Samsung smartphones rank first, followed by Apple and ASUS phones. The query “apple” returns all 21 Apple products, including MacBooks whose titles lack the word. A nonsense query prints “No matching products found”, and one- or two-letter queries and bare numbers are not matched against random words. The console also prints “Showing top 10 of 77 matching products” so the user knows more results exist.



```
PS C:\Users\tallu\Downloads\Product_Search_Recommendation_System_expoo\Product_Search_Recommendation_System> javac -d .\out .\src\*.java
PS C:\Users\tallu\Downloads\Product_Search_Recommendation_System_expoo\Product_Search_Recommendation_System> java -cp .\out Main
```

```
=====
PRODUCT SEARCH & RECOMMENDATION
=====
```

Corpus files loaded: 300

KMP / Rabin-Karp for pattern matching.  
Levenshtein for spelling recovery.  
Edmonds-Karp for recommendations.

Type exit to close the system.

Enter search query: Samsng phone

```
=====
SEARCH RESULTS
=====
```

Query : Samsng phone  
Did you mean : samsung phone?  
Spelling : samsng -> samsung (Levenshtein edit distance = 1)  
Algorithm : Levenshtein (spelling) -> KMP (matching)

1. Samsung Galaxy S24 Ultra

Price : INR 1,29,999  
Rating : 4.6/5  
Brand : Samsung  
Category : Smartphone  
Info : The Samsung Galaxy S24 Ultra is Samsung's flagship Android smartphone, built around a titanium frame and a flat 6.8-inch...

2. Samsung Galaxy S25 Ultra

Score : 830  
Price : INR 1,29,999  
Rating : 4.7/5  
Brand : Samsung  
Category : Smartphone  
Info : The Samsung Galaxy S25 Ultra is a Samsung mobile phones designed for users who need dependable performance, practical fe...

3. Samsung Galaxy S24 FE

Score : 830  
Price : INR 59,999  
Rating : 4.6/5  
Brand : Samsung  
Category : Smartphone  
Info : The Samsung Galaxy S24 FE is a smartphone from Samsung, designed for dependable everyday performance and practical use.

4. Samsung Galaxy Z Flip6

Score : 830

```

9. Asus ROG Phone 9 Pro
  Score   : 540
  Price   : INR 1,19,999
  Rating  : 4.7/5
  Brand   : ASUS
  Category : Smartphone
  Info    : The Asus ROG Phone 9 Pro is a gaming phone from Asus, designed for dependable everyday
           performance and practical use.

10. Apple iPhone 16
  Score   : 540
  Price   : INR 79,900
  Rating  : 4.6/5
  Brand   : Apple
  Category : Smartphone
  Info    : The Apple iPhone 16 is a Apple mobile phones designed for users who need dependable per
           formance, practical features, and...

Showing top 10 of 77 matching products.
=====
Would you like product recommendations? (Y/N): n

```

Fig. 5.1 Console output: start-up (300 files loaded) and the search “samsng phone” with spelling correction

### 5.3 Recommendation Results

Table 5.2 shows the recommendations for “samsng phone”. Six Samsung phones were among the 30 candidates, but the brand limit of two admitted only the best two; the console listed the others as “brand limit reached”. The constraint check printed Total 6/6, Category Smartphone 6/6 and 2/2 for Samsung, Apple and ASUS, followed by “All constraints satisfied”.

**Table 5.2 Recommendations for “samsng phone” (limits: total 6, category 6, brand 2)**

| # | Product                  | Brand   | Score |
|---|--------------------------|---------|-------|
| 1 | Samsung Galaxy S24 Ultra | Samsung | 880   |
| 2 | Samsung Galaxy S25 Ultra | Samsung | 830   |
| 3 | Apple iPhone 16 Pro      | Apple   | 540   |
| 4 | Apple iPhone 16 Pro Max  | Apple   | 540   |
| 5 | Asus ROG Phone 9 Pro     | ASUS    | 540   |
| 6 | Asus ROG Phone 8         | ASUS    | 540   |

For the query “apple” every candidate has the same brand, so the default limits could fill only 2 of 6 slots; the system reported this, switched to 2 per category and 6 per brand, and returned one Apple product from each of six categories (two smartphones, a laptop, a monitor, a wearable and an audio product). The query “waterproof” matched only three products (one Apple, two Samsung), so only three were recommended: the list is never padded to six.



## 5.4 Trie Prefix Search Results

**Table 5.3 Trie prefix search**

| Prefix      | Products found | First suggestions printed                                                    |
|-------------|----------------|------------------------------------------------------------------------------|
| app         | 21             | Apple iPhone 16, Apple Watch Ultra 2, Apple iPhone 16 Pro, ...               |
| samsung gal | 15             | Samsung Galaxy S25 Ultra, Samsung Galaxy Buds3 Pro, Samsung Galaxy Fit3, ... |
| xyz         | 0              | No matching products found                                                   |

The prefix “app” also returns MacBooks. This is by design, not a defect: the Trie indexes the Search Index section as well as the name, and the MacBook records list the word “apple” there. Ten suggestions are printed and the console states the total (“Showing 10 of 21 matching products”). For several words the product sets are intersected, so “samsung gal” returns only products having words starting with both.



```

Match-count consistency: PASS
Timing observation: KMP was faster in this benchmark run.
Note: timings depend on JVM warm-up, hardware, and system load;
      a small timing difference is not proof that one algorithm is always faster.
=====

```

Fig. 5.4 Console output of the /time benchmark for the pattern “sam”

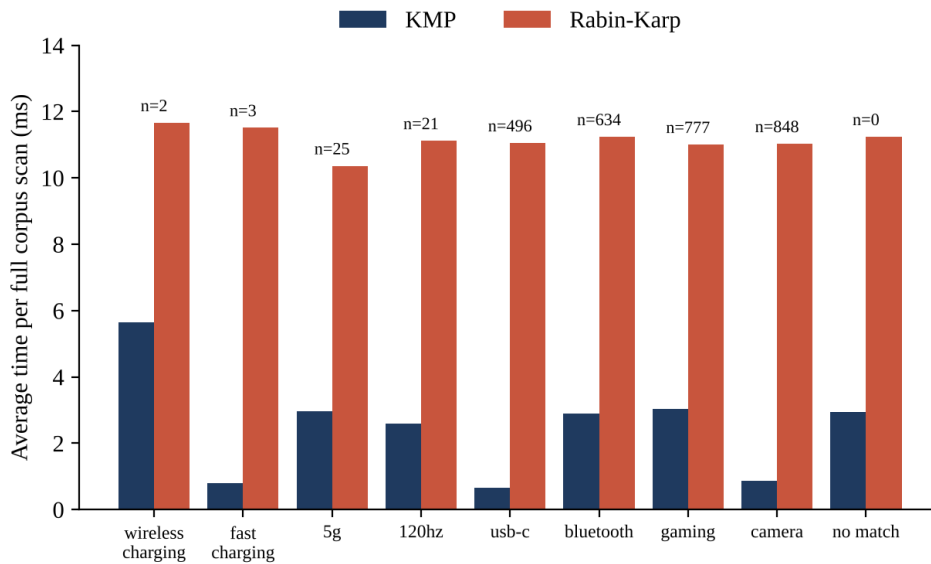


Fig. 5.5 Average scan time of KMP and Rabin–Karp over the corpus (n = number of occurrences)

KMP was faster in every row. The gap is a property of this implementation, not of the algorithms: the Rabin–Karp loop performs two 64-bit modulo operations per character, while KMP only compares characters. Rabin–Karp’s strength is searching many patterns at once, which this single-phrase benchmark does not exercise. KMP times vary (about 0.6 to 5.7 ms) with JVM warm-up and load; only the equality of the counts is deterministic.

## 5.6 Correctness Verification

Besides the 184 checks of TestRunner, the algorithms were compared with independent implementations (Table 5.5). All comparisons agreed. Compilation was also tested with --release 8, 11, 17 and 21; run-time behaviour on JDK 8, 11, 17 and on Windows was not tested and is not claimed.

**Table 5.5 Verification of the algorithms**

| Component             | Reference used                                 | Cases               | Outcome              |
|-----------------------|------------------------------------------------|---------------------|----------------------|
| KMP and Rabin–Karp    | Python regular expressions (overlapping count) | 30,008              | All equal            |
| Levenshtein           | Recursive definition in Python                 | 20,008 pairs        | All equal            |
| Edmonds–Karp          | networkx maximum flow                          | 4,000 random graphs | All equal            |
| Recommendation limits | Brute-force search over all subsets            | 600 cases           | All equal            |
| Trie prefix listing   | Sorted set (TreeSet) of the same words         | 9,000 queries       | All equal            |
| Whole system          | TestRunner (exit code 0)                       | 184 checks          | 184 passed, 0 failed |

```
== Levenshtein
== Trie
== Edmonds-Karp max flow
== Recommendation policy (max flow)
== Corpus consistency
== ProductDocument parsing
== Search engine
== Console flow (search + recommendations)
== Trie prefix search (corpus)
== Levenshtein fallback search
== Benchmark counts (/time)

=====
Checks passed: 184
Checks failed: 0
RESULT: ALL TESTS PASSED
=====
```

**Fig. 5.6 Automated test suite result (TestRunner)**

## 6. CHALLENGES & LIMITATIONS

### 6.1 Issues Faced During Development

The original project was reviewed by compiling and running it and by checking it against the references of Section 5.6. The five core algorithms were correct; most defects were in the data and in the ranking around them (Table 6.1).

**Table 6.1 Principal issues found during review**

| # | Issue                                                                             | Effect                                            |
|---|-----------------------------------------------------------------------------------|---------------------------------------------------|
| 1 | 21 records had a wrong category (headphones and a smartwatch filed as Smartphone) | “phone” returned headphones                       |
| 2 | Category guessed from the product name overrode the Category field                | A laptop treated as a monitor                     |
| 3 | Brand-only and category-only matches were discarded                               | “apple” missed every MacBook                      |
| 4 | Substring matching                                                                | “n” matched every product; “pro” matched “ProArt” |
| 5 | Poor spelling suggestions                                                         | “ultr” → “ult”; “amoled” → “oled”; “128” → “120”  |
| 6 | Crash when input ended at the Y/N prompt                                          | NoSuchElementException                            |
| 7 | Four products stored twice; corpus had 257, not 300 files                         | Duplicate results                                 |
| 8 | Rupee sign read with the platform encoding                                        | Prices printed as “INR ?129,999” on Windows       |
| 9 | Benchmark printed one “expected” count for both algorithms                        | Consistency not checkable                         |

### 6.2 Solutions Applied

- **Data:** categories were set from each record’s Subcategory (21 records), duplicate pairs removed, “Asus” unified to “ASUS” (11 records), and 43 new distinct products added so the corpus has exactly 300 files.
- **Matching and spelling:** whole-word matching replaced substring matching; brand and category count as real matches; words in the vocabulary, with digits or under three letters are never rewritten.
- **Robustness:** every input read is guarded, a missing folder prints a hint, and the rupee sign is written as a `\u20B9` escape so all sources are ASCII (compile with `-encoding UTF-8`).
- **Verification:** each algorithm prints its own benchmark count, one shared PatternMatching class is used, and TestRunner (184 checks) keeps every fix covered; the corpus checks fail on the original data.

The displayed corpus count is the length of the array returned by the loader, not a constant; it showed 300 for the full folder and 10 for a folder of 10 files.

### 6.3 Limitations of the System

- **Sample data:** the 43 added records and many descriptions use generic text with illustrative prices and ratings; they must not be presented as verified specifications.
- **Brand-limit policy:** the brand limit is relaxed when the default limits cannot fill six slots (for example “apple”); a strict mode needs the two fallback tiers removed.
- **Queries:** glued queries such as “googlepixel9” are not split into words; spelling recovery works word by word only.
- **Recommendations** are a diverse subset of the best 30 search results, not products from outside the query.
- **Benchmark and platforms:** timings depend on JVM warm-up and load; run-time tests were done on JDK 21 on Linux only.

## **7. CONCLUSION & FUTURE WORK**

### **7.1 Conclusion**

The project delivers a working console Product Search and Recommendation System in which KMP, Rabin–Karp, Levenshtein distance, a Trie and Edmonds–Karp cooperate on a corpus of 300 products. Search is whole-word and explainable, spelling mistakes are corrected conservatively, prefix suggestions are alphabetical, and the recommendation list always respects its limits because the limits are the capacities of a flow network. KMP and Rabin–Karp agree on every tested input. Every claim in this report is backed by an executed test, and the limitations (sample data, the brand-limit policy, platform coverage) are stated openly.

### **7.2 Future Work**

- Replace the sample specifications and prices with verified data and a database; add a strict-limit mode with user-chosen limits.
- Use Aho–Corasick or multi-pattern Rabin–Karp to match many phrases at once.
- Use price and rating in the flow network (for example min-cost max-flow) and add filters and sorting.
- Add personalisation and a simple front end if the course scope allows, and run the tests on several JDKs and on Windows.

### **7.3 Skills Learned During Development**

- Implementing KMP, Rabin–Karp, Levenshtein, Trie and Edmonds–Karp from first principles and analysing their complexity.
- Modelling a practical selection problem (diverse recommendations) as maximum flow and designing an explainable ranking function.
- Writing self-checking tests, comparing against independent references and reporting measurements honestly.
- Handling text encodings and data consistency in a real corpus, and writing technical documentation.

## REFERENCES

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
- [2] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast pattern matching in strings,” *SIAM J. Comput.*, vol. 6, no. 2, pp. 323–350, 1977.
- [3] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” *IBM J. Res. Develop.*, vol. 31, no. 2, pp. 249–260, 1987.
- [4] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [5] E. Fredkin, “Trie memory,” *Commun. ACM*, vol. 3, no. 9, pp. 490–499, 1960.
- [6] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” *J. ACM*, vol. 19, no. 2, pp. 248–264, 1972.
- [7] L. R. Ford Jr. and D. R. Fulkerson, “Maximal flow through a network,” *Canadian J. Math.*, vol. 8, pp. 399–404, 1956.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [9] D. Gusfield, *Algorithms on Strings, Trees, and Sequences*. Cambridge, U.K.: Cambridge Univ. Press, 1997.
- [10] R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” *J. ACM*, vol. 21, no. 1, pp. 168–173, 1974.
- [11] G. Navarro, “A guided tour to approximate string matching,” *ACM Comput. Surveys*, vol. 33, no. 1, pp. 31–88, 2001.
- [12] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015.
- [13] Oracle Corporation, “Java Platform, Standard Edition Documentation.” [Online]. Available: <https://docs.oracle.com/en/java/javase/>